

תרגיל 4 – Word Embeddings

מבוא

בתרגיל הזה נתנסה בכמה שימושים שיש ל-word embeddings: דמיון מילים, מודלי שפה וסיווג. גם בתרגיל זה נשתמש בקורפוס הכנסת, כלומר בקובץ jsonl שיצרתם בתרגילים הקודמים.

חלק א': יצירה של מודל Word2Vec ואימונו על קורפוס הכנסת

בחלק זה, אתם תבנו מודל word2vec שיתאמן על המשפטים מקורפוס הכנסת, ויוכל ליצור בעזרתם word embeddings לכל מילה בקורפוס. כלומר, וקטורים שמייצגים כל מילה בקורפוס.

לשם כך, תשתמשו בספרייה gensim ותייבאו ממנה את המודל Word2Vec באופן הבא:

```
from gensim.models import Word2Vec
```

א. הכינו למודל את המשפטים (לאחר טוקניזציה) מתוך קורפוס הכנסת. לשם כך, הסירו מהמשפטים טוקנים שאינם מילים (סימני פיסוק, מספרים וכד') והכינו מהם רשימה של רשימות של טוקנים. כלומר, כל משפט יהיה רשימה של הטוקנים הנמצאים בו, והם יוספו לרשימה המכילה את כל הרשימות האלו, שתהיה רשימת כל המשפטים. דוגמה לאיך הרשימה אמורה להיראות:

```
# Example: tokenized_sentences = [['היום', 'הוא', 'יום', 'נפלא'], ['אני', 'אוהב', 'אורי', 'היום']]
```

ב. על מנת לבנות מודל Word2Vec המייצר וקטורי word embeddings בגודל 50, מתוך משפטי הכנסת, ניתן להשתמש בפקודה הבאה:

```
model = Word2Vec(sentences=tokenized_sentences, vector_size=50, window=5, min_count=1)
```

היכנסו לתיקוד המודל, הבינו מה המשמעות של כל ארגומנט שנתנו לו (vector_size, window, min_count) ובחרו את הערכים של ארגומנטים אלו בעצמכם. **הסבירו את בחירתכם בדו"ח.**

כדאי גם לשמור את המודל כדי שלא תצטרכו לאמן אותו מחדש בכל ריצה:

```
model.save("knesset_word2vec.model")
```

ג. כעת אתם יכולים להשתמש במודל באופן הבא:

```
word_vectors = model.wv
print(word_vectors['ישראל']) # Get the vector for the word
```

ענו על השאלות הבאות:

1. הסבירו מה המשמעות, ומה היתרונות והחסרונות של הגדלת והקטנת גודל הוקטור - vector_size.
2. הסבירו מה הבעיות שיכולות לעלות משימוש במודל הני"ל, שאומן על הקורפוס שלנו. התייחסו בתשובתכם לאופן שבו יצרנו את הקורפוס, לגודל שלו ולשימושים פוטנציאליים של המודל.

חלק ב': דמיון בין מילים

וקטורי word embeddings אמורים לייצג את משמעות המילים. לכן, עבור מילים דומות במשמעות, נצפה שהמרחק בין הוקטורים שלהן יהיה קצר. בחלק זה נבדוק אם זה באמת מתקיים.

- א. לפניכם רשימה של מילים, עבור כל מילה ברשימה עליכם למצוא את 5 המילים הכי קרובות אליה (לא כולל עצמה כמובן). אתם יכולים לעשות זאת ע"י שימוש בפונקציה similarity שמחשבת את ה cosine similarity בין שני וקטורי מילים.

```
similarity_score = model.wv.similarity('word1', 'word2')
```

רשימת המילים:

1. יום
2. אישה
3. דרך
4. ארוך
5. תוכנית
6. אוהב
7. אסור
8. איתן
9. זכות

הדפיסו את המילים והמרחקים לקובץ בשם "kneset_similar_words.txt" באופן הבא (ללא המשולשים <>):

<first given word>: (<first word>, <similarity score>), ..., (<last word>, <similarity score>)

...

<last given word>: (<first word>, <similarity score>), ..., (<last word>, <similarity score>)

ב. צרו לכל משפט בקורפוס sentence embeddings ע"י ממוצע משוקלל של וקטורי המילים במשפט. התעלמו מטוקנים Word2Vec לא מכיר בחישוב זה (אלו שהסרנו בחלק א'). החישוב יעשה באופן הבא:

$$\frac{\sum_{i=1}^k v_i}{k}$$

כאשר v_i הוא וקטור המילה ה- i במשפט ו- k הוא מספר המילים שהחשבנו במשפט.

ג. בחרו 10 משפטים מהקורפוס, המכילים לפחות 4 טוקנים תקינים (מילים), ועבור כל אחד מצאו בעזרת [cosine similarity](#) את המשפט הכי קרוב אליו.

הדפיסו את המשפטים לקובץ בשם "knesset_similar_sentences" באופן הבא:

<first chosen sentence>: most similar sentence: <most similar sentence>

...

<last chosen sentence>: most similar sentence: <most similar sentence>

הערות:

1. הדפיסו את המשפטים **המקוריים** מהקורפוס שלכם, עם רווח בין הטוקנים (כולל סימני פיסוק וכדו').

2. השתדלו לבחור משפטים שלדעתכם מציגים נאמנה את היכולות של המודל על הקורפוס שלכם.

ד. לפניכם 5 משפטים שבכל אחד מהם ישנן מילים המסומנות באדום. השתמשו בפונקציה most_similar כדי למצוא מילה קרובה למילה המסומנת באדום והחליפו אותה בה. פונקציה זו מקבלת גם ארגומנטים של negative ו positive שמיועדות לביצוע אריתמטיקה וקטורית, ובעזרתם ניתן לכוון את המודל לבחור במילה יותר מתאימה. בדקו האם אתם מצליחים להחליף את המילים האדומות במילים אחרות, כך שהמשפט נשאר תקין, הגיוני ובעל משמעות דומה. **הסבירו בדו"ח מה ניסיתם, מה עשיתם, מה קיבלתם והאם הצלחתם במשימה.**

דוגמה:

נניח ויש לנו את המשפט הבא:

"היום בבוקר **מהנדסת** הציגה את המודל שבנתה"

ונניח והשתמשנו בפונקציה most_similar כדי למצוא מילה שתחליף את המילה האדומה "מהנדסת" במילה מספיק קרובה שתשמור על המשמעות וגם על נכונות תחבירית. תחילה, ניסיתם להשתמש בפונקציה כך:

```
similar_word = model.wv.most_similar('מהנדסת', topn=1)
```

ונניח שקיבלתם חזרה את המילה "מהנדס"

מילה זו לא מסתדרת מספיק טוב במשפט כי הוא מנוסח בלשון נקבה, אז ננסה שוב, הפעם בעזרת הארגומנטים positive, negative:

```
similar_word = model.wv.most_similar(positive=['מהנדסת', 'אישה'],
negative=['גבר'], topn=1)
```

כעת המודל החזיר את המילה "הנדסאית" שמתאימה למשפט בצורה טובה יותר.

באופן דומה, במשפטים הבאים נסו להחליף את המילים האדומות במילה הכי טובה שתצליחו:

1. **בשעות** הקרובות נפתח את הדיון **בסעיף** הבא שעל סדר היום
2. **אבקש** מהנוכחים **להתיישב** כדי שנוכל להתחיל.
3. אנו מודים **לצוות** המקצועי על **עבודתו**.
4. **הנושא** יועבר להמשך **טיפול** בוועדת המשנה.
5. **ההצעה הובאה** להצבעה **ואושרה**.

הדפיסו את הפלט עם המילים המחליפות שבחרתם, לקובץ בשם "red_words_sentences.txt" בפורמט הבא:

1: <original sentence>: <new sentence where the red words were replaced with the new words>

replaced words: (<original red word>:<new replaced word>),...(<original red word>:<new replaced word>)

...

5: <original sentence>: <new sentence where the red words were replaced with the new words>

replaced words: (<original red word>:<new replaced word>),...(<original red word>:<new replaced word>)

הערות:

1. מותר לכם לקחת אחת מתוך שלושת המילים הכי דומות שהמודל מחזיר topn=3.
2. אף על פי שלרוב זה נחוץ וזאת נקודת התחלה טובה, אינכם חייבים להשתמש במילה האדומה עצמה כדי להגיע למילה מחליפה. למשל, אם המילה האדומה לא קיימת בקורפוס שלכם אבל מילה קרובה כן, או אם אתם חושבים שבעזרת מילה אחרת **שקרובה אל המילה האדומה במשמעות** תצליחו להביא את המודל להחזיר מילה מחליפה מתאימה יותר, מותר לכם לעשות זאת.

ענו על השאלות הבאות:

1. האם המילים הכי קרובות שקיבלתם בסעיף א' תואמות את הציפיות שלכם? הסבירו. גם אם תאמו לציפיות וגם אם לא, נסו להסביר מדוע זה עבד או לא עבד טוב.

2. אם ניקח שתי מילים שנחשבות להפכים (antonyms), למשל "אהבה" ו"שנאה", או "קל" ו"כבד". האם היינו מצפים שהמרחק בין שני וקטורי המילים שלהן יהיה קצר או ארוך? הסבירו.
3. מצאו שלושה זוגות של מילים שנחשבות להפכים (antonyms) הקיימות בקורפוס שלנו ובדקו את המרחק ביניהן. האם הציפייה שלכם מסעיף 2 מתקיימת עבורן עם המודל שבניתם?
4. האם המשפטים הכי קרובים בסעיף ג' תאמו לציפיות שלכם? הסבירו. גם אם תאמו לציפיות וגם אם לא, נסו להסביר מדוע זה עבד או לא עבד טוב.

חלק ג': סיווג

אמנו מודל KNN בדומה לזה שמימשתם בתרגיל בית 3, על מנת לסווג בין משפטים של הדוברים שבחרתם בתרגיל הקודם (סיווג בינארי). הפעם, במקום וקטורי המאפיינים שהשתמשתם בהם בתרגיל 3, השתמשו ב-sentence embeddings, כפי שהגדרנו בחלק ב' סעיף ב'. השתמשו בשיטת 5-fold cross validation כפי שעשיתם בתרגיל הקודם. הפיקו [classification report](#) וצרפו אותו לדו"ח.

ענו על השאלות הבאות:

1. האם עבור אותם פרמטרים ותנאים שהשתמשתם בהם בתרגיל 3 קיבלתם תוצאות טובות יותר או פחות עבור וקטור המאפיינים הנ"ל? **הסבירו** מדוע לדעתכם זה קרה.

חלק ד': שימוש במודלי שפה גדולים

כפי שנלמד בהרצאות הבאות, מודלי שפה גדולים (LLM- Large language models) כמו GPT-3/4, BERT, RoBERTa ועוד, תפסו מקום חשוב מאוד בעולם עיבוד השפות הטבעיות בשנים האחרונות. אלו הם מודלים שבנויים מרשתות נוירונים עמוקות מאוד ומאומנים על הרבה מאוד טקסט. הם לומדים "להבין" את המשמעויות העמוקות והמורכבויות שיש בשפה והם מסוגלים לבצע בדיוק גבוה משימות רבות בתחום ה-NLP כגון חיזוי המילה הבאה במשפט, יצירה (generation) של טקסט קוהרנטי ורלוונטי, תרגום, מענה על שאלות ועוד.

לב ההצלחה של מודלים אלו והבסיס שבעזרתו חלקם מסוגלים לבצע הרבה מהמשימות האלו, הוא הדרך שבה הם מצליחים לייצר word embeddings. בניגוד ל-Word2Vec, ה-word embeddings שמודלים אלו מייצרים הם תלויי קונטקסט.

מודלי word embeddings לא קונטקסטואלים יפיקו לאותה מילה תמיד את אותו הוקטור, אף על פי שיכולה להיות לה משמעות שונה לחלוטין בהקשרים שונים. למשל: המילה "חשב" במשפט "הוא **חשב** שהרעיון היה מצויין" לעומת במשפט "הוא **חשב** במקצועו".

מודלי שפה גדולים מתייחסים לקונטקסט שסביב המילה כדי לייצר לה וקטור המתאים למשמעות שלה באותו ההקשר.

כדי להתנסות במודל הזה, אנחנו נשתמש במודל שנקרא [DictaBERT](#). זהו מודל מבוסס BERT שאומן על הרבה מאוד טקסטים בעברית ומסוגל לחזות בדיוק יחסית גבוה מילים ממוסכות (masked) במשפט.

השתמשו בקובץ המשפטים הממוסכים שיצרתם בתרגיל בית 2 `masked_sampled_sents.txt`, החליפו את הכוכביות [*] בטוקן [MASK] כפי שdictaBert מצפה, עקבו אחר ההוראות בדף של המודל ותחזו בעזרתו את המילים החסרות במשפטים.

צרו קובץ פלט, בשם `dictabert_results.txt` בפורמט דומה לזה שעשיתם בתרגיל בית 2, רק עם החיזויים של המודל הזה, באופן הבא:

```
masked_sentence: <first masked sentence as appeared in the your masked_sampled_sents.txt file>
dictaBERT_sentence: < The sentence with the generated tokens as was produced by DictaBERT >
dictaBERT tokens: <A list of the generated tokens, separated by a comma (“,”)>
<and so on>
```

ענו על השאלות הבאות:

1. האם קיבלתם משפטים הגיוניים? מבחינת התוכן, קוהרנטיות ומבחינה תחבירית.
2. האם קיבלתם השלמות קרובות למילים החסרות האמיתיות? פרטו
3. השוו את התוצאות שקיבלתם עכשיו לאלו שקיבלתם בתרגיל בית 2. האם יש שיפור בתוצאות לדעתכם?
4. האם יש משפטים שעבורם המודל עבד פחות טוב? אם כן ואם לא, הסבירו מה לדעתכם הסיבה לכך.
5. באילו מקרים לדעתכם יש למודל סיכוי גבוה יותר להכשל או להחזיר תוצאה פחות טובה? מדוע?

ספריות מותרות לשימוש

אתם יכולים להשתמש ב

Pandas, Numpy, scikit-learn, genism, transformers*, torch*

(*) רק עבור חלק ד')

ובכל ספרייה סטנדרטית של python.

אתם יכולים לחפש שם של ספרייה ב <https://docs.python.org/3/library/index.html> על מנת לבדוק אם זו

ספרייה סטנדרטית. לא יהיה מענה על שאלות לגבי שימוש בספריות ספציפיות.

- למען הסר ספק, json היא ספרייה סטנדרטית של python.
- מומלץ להשתמש עבור כל פרויקט בסביבה וירטואלית virtual environment חדשה משלו על מנת להיות בטוחים שאתם משתמשים רק בספריות מותרות ולמנוע קונפליקטים עם ספריות קודמות שהתקנתם בעבר. ראו מצגת על כך במודל.

הערות כלליות

1. על הקוד שלכם להיות מסוגל להתמודד עם שגיאות בכל שלב בתהליך ולא לקרוס. השתמשו ב-Try Except blocks לפי הצורך.
2. שימו לב, בבדיקת תרגילי הבית בקורס ניתן משקל גדול מהניקוד הן על הדו"ח, ההסברים והידע שהפגנתם בחומר הנלמד והן על הקוד, אופן המימוש, יעילותו, קריאותו ועמידותו. בפרט, הרבה מהבדיקות הן אוטומטיות ולכן עליכם להקפיד על קוד תקין שרץ ללא שגיאות ועל עמידה מדויקת בפלט הנדרש וביתר ההנחיות.
3. ניתן לשאול שאלות על התרגיל בפורום המיועד במודל. למעט מקרים אישיים מיוחדים, אין לשלוח שאלות הקשורות לתרגיל הבית במייל.
4. על אחריותכם לעקוב אחר הודעות הקורס במודל (בלוח הודעות ובפורום) ולהיות מעודכנים במידה ויהיו שינויים בהנחיות.

אופן ההגשה

1. ההגשה היא בזוגות בלבד.
2. עליכם להגיש קובץ zip בשם `hw4_<id1>_<id2>.zip` (כאשר `<id1>`, `<id2>` הם מספרי תעודות זהות של הסטודנט הראשון והשני בהתאמה), המכיל את הקבצים הבאים:
 - a. קובץ `python kneset_word2vec.py` בשם `kneset_word2vec.py` המכיל את כל הקוד הנדרש כדי לממש את חלקים א'-ב'.
 - i. - הקלט לקובץ יהיה נתיב לקובץ `jsonl` של קורפוס הכנסת ונתיב לתיקייה בה תשמרו את מודל `word2vec` שיצרתם וגם תייצרו את שלושת קבצי הפלט (התואמים לקבצים `d,e,f`).
 - הפלט צריך להיות המודל והקבצים התואמים `d,e,f` שמורים בתיקייה שהתקבלה כקלט.
 - ii. על הקובץ לרוץ תחת הפקודה (ללא הסימונים `<>`):

`python kneset_word2vec.py <path/to/corpus_file.jsonl> <path/to/output_dir>`

- b. קובץ `python kneset_word2vec_classification.py` בשם `kneset_word2vec_classification.py` המכיל את כל הקוד הנדרש כדי לממש את חלק ג'.
- i. - הקלט לקובץ יהיה נתיב לקובץ `jsonl` של קורפוס הכנסת, נתיב למודל `word2vec` (כפי ששמרתם בחלק א')
- הפלט יהיה הדפסה למסך של `classification report` (כפי שתואר בחלק ג').
- ii. על הקובץ לרוץ תחת הפקודה (ללא הסימונים `<>`):

`python kneset_word2vec_classification.py <path/to/corpus_file.jsonl> <path/to/kneset_word2vec.model>`

- c. קובץ `Python kneset_dictabert.py` בשם `kneset_dictabert.py` המכיל את כל הקוד הנדרש כדי לממש את חלק ד'.
- i. - הקלט לקובץ יהיה נתיב לקובץ `masked_sampled_sents.txt` ונתיב לתיקייה בה תייצרו את קובץ הפלט `dictabert_results.txt`

- הפלט צריך להיות הקובץ dictabert_results.txt שמור בתיקייה שהתקבלה כקלט.

ii. על הקובץ לרוץ תחת הפקודה :

`python kneset_dictabert.py <path/to/ masked_sampled_sents.txt> <path/to/output_dir>`

d. קובץ text בשם kneset_similar_words.txt כפי שתואר בחלק ב' סעיף א'.

e. קובץ text בשם kneset_similar_sentences.txt כפי שתואר בחלק ב' סעיף ג'.

f. קובץ text בשם red_words_sentences.txt כפי שתואר בחלק ב' סעיף ד'.

g. קובץ text בשם dictabert_results.txt כפי שתואר בחלק ד'.

h. קובץ jsonl של קורפוס הכנסת איתו עבדתם.

i. קובץ text בשם masked_sampled_sents.txt שיצרתם בתרגיל בית 2.

j. קובץ text בשם original_sampled_sents.txt שיצרתם בתרגיל בית 2.

k. קובץ PDF בשם `<id1>_<id2>_hw4_report.pdf` ובו דו"ח המפרט על הקוד, על ההחלטות

שקיבלתם במהלך העבודה על התרגיל ומענה על כל השאלות בכל החלקים.

אל תשכחו לציין בתחילת הדו"ח את שמותיכם בעברית ותעודות הזהות שלכם.

יש להקפיד על עבודה עצמית, צוות הקורס יתייחס בחומרה להעתקות או שיתופי קוד, כמו גם שימוש בכלי AI דוגמת chatGPT.

ניתן לשאול שאלות על התרגיל בפורום הייעודי לכך במודל.

יש להגיש את התרגיל עד לתאריך 08.01.26 בשעה 23:59.

בהצלחה!